

Institut Supérieure d'Informatique et de
Multimédia de Sfax



Atelier de Programmation

Chapitre 5

Les tableaux

Enseignant : Ramzi ZOUARI

Filière : P-LSI

Année universitaire : 2025-2026

Plan

1. Introduction
2. Déclaration d'un tableau
3. Manipulations sur les tableaux
4. Passage de paramètres avec les tableaux

1. Introduction

- Un tableau est une variable qui se compose d'un certain nombre de données de même type, rangées en mémoire les une après les autres.
- Le contenu d'un tableau peut être:
 - **type élémentaire**: int, char, long, float, double, ...
 - **pointeur**: adresse mémoire d'une autre donnée
 - **tableau**: chaque élément du tableau peut être à son tour un tableau
 - **structure**: un type de données complexe regroupant des variables de type simple
- **Remarque**: Dans ce cours, on s'intéresse uniquement aux tableaux **unidimensionnels**, c.à.d chaque élément du tableau n'est pas lui-même un tableau

2. Déclaration d'un tableau

- La déclaration d'un tableau unidimensionnel, plus précisément une variable de type tableau, admet les syntaxes suivantes:

Syntaxe 1

```
<type> <nom_du_tableau> [nombre elements];
```



Exemple 1

```
int main(){  
    int t [4];  
    return 0;  
}
```

Syntaxe 2

```
<type> <nom_du_tableau> [] = {elem1, elem2,..., };
```



Exemple 2

```
int main(){  
    int t [] = {10, 22, 36, 50};  
    return 0;  
}
```

3. Manipulations sur les tableaux

- L'accès à un élément du tableau se fait à travers son indice
 - Les indices d'un tableau de taille **N** varient entre **0** et **N-1**
 - La syntaxe **T[2]** permet d'accéder à l'élément d'indice **2** dans le tableau **T**
 - Chaque élément du tableau est caractérisé par son **indice** dans le tableau et par son **adresse mémoire**
 - La **taille** mémoire occupée par un élément dépend de son **type**

Exemple

```
int main(){  
    int T [] = {10, 22, 36, 50};  
    return 0;  
}
```



contenu →	T[0]	T[1]	T[2]	T[3]
indices →	0	1	2	3
	10	22	36	50
Adresses →	0x2000	0x2004	0x2008	0x200C

3. Manipulations sur les tableaux

- Pour remplir et afficher les données d'un tableau T, on utilise les instructions suivantes:
 - `scanf("%d", &T[3])` → Lire une donnée de type entier et la stocker dans l'adresse de l'indice 3 du tableau T
 - `printf("%d", T[2])` → Afficher la valeur stockée à l'indice 2 du tableau T

Exemple

```
#include<stdio.h>
int main(){
    int n=4, T[n];
    // Remplissage de T par les valeurs 2 - 38 - 47 - 41
    for (int i=0; i<n; i++){
        printf("T[%d]= ", i);
        scanf("%d", &T[i]);
    }
    // Affichage de T
    for (int i=0; i<n; i++)
        printf("\nT[%d]=%d -- Adresse Memoire=%x", i, T[i], &T[i]);
    return 0;
}
```



Résultat

T[0]=2	-- Adresse Memoire=15ff740
T[1]=38	-- Adresse Memoire=15ff744
T[2]=47	-- Adresse Memoire=15ff748
T[3]=41	-- Adresse Memoire=15ff74c

3. Manipulations sur les tableaux

- Toutes les opérations applicables sur les variables restent valides pour les tableaux

	0	1	2	3
T	10	22	36	50
	0x2000	0x2004	0x2008	0x200C

Instruction	Explication	Résultat (i=2)			
T[i]=4	-	10	22	4	50
T[++i]=25	i = i + 1; T[i] = 25;	10	22	36	25
T[i++]=25	T[i] = 25; i = i + 1;	10	22	25	50
T[i]++	T[i] = T[i] + 1;	10	22	37	50
++T[i]	T[i] = T[i] + 1;	10	22	37	50
T[i]+=4	T[i] = T[i] + 4;	10	22	40	50

4. Passage de paramètres avec les tableaux

- Comme dans le cas des variables de type simple, une variable de type tableau peut être envoyée à un sous programme en utilisant le passage par valeur ou par adresse

Passage par valeur

```
#include<stdio.h>
int n = 4;

void afficher(int tab[]){
    for (int i=0; i<n; i++)
        printf("\t%d", tab[i]);
}

int main(){
    int T [] = {10, 22, 36, 50};
    afficher(T);
    return 0;
}
```

10	22	36	50
----	----	----	----

Passage par valeur

```
#include<stdio.h>
int n = 4;
void afficher(int tab[]){
    for (int i=0; i<n; i++)
        printf("\t%d", tab[i]);
}
void modifier(int tab[], int idx, int val){
    tab[idx] = val;
}
int main(){
    int T [] = {10, 22, 36, 50};
    modifier(T, 2, 10);
    afficher(T);
    return 0;
}
```

10	22	10	50
----	----	----	----

4. Passage de paramètres avec les tableaux

- **Remarque:** En C, un sous programme peut retourner une variable de type simple, mais il ne permet pas de retourner une variable de type tableau

Passage par valeur

```
#include<stdio.h>
int n = 4;
void afficher(int tab[]){
    for (int i=0; i<n; i++)
        printf("\t%d", tab[i]);
}
void modifier(int tab[], int idx, int val){
    tab[idx] = val;
}
int main(){
    int T [] = {10, 22, 36, 50};
    modifier(T, 2, 10);
    afficher(T);
    return 0;
}
```



Passage par valeur

```
#include<stdio.h>
int n = 4;
void afficher(int tab[]){
    for (int i=0; i<n; i++)
        printf("\t%d", tab[i]);
}
int [] modifier(int tab[], int idx, int val){
    tab[idx] = val;
    return tab;
}
int main(){
    int T [] = {10, 22, 36, 50}, T1[4];
    T1 = modifier(T, 2, 10);
    afficher(T);
    return 0;
}
```

4. Passage de paramètres avec les tableaux

- Dans le passage par adresse, on n'utilise pas le symbole `&` dans l'appel des fonctions car la variable de type tableau n'a pas une seule adresse comme le cas des variables de type simple.
- **Solution 1:** Faire l'appel au sous programme en utilisant l'adresse du 1^{er} élément du tableau T

Passage par adresse

```
#include<stdio.h>
int n=4;
void afficher(int *tab){
    for (int i=0; i<n; i++)
        printf("\t%d", *(tab+i));
}
void modifier(int tab[], int idx, int val){
    tab[idx] = val;
}
int main(){
    int T [] = {10, 22, 36, 50};
    modifier(&T, 2, 10);
    afficher(&T);
    return 0;
}
```

Erreur



Passage par adresse (Solution 1)

```
#include<stdio.h>
int n=4;
void afficher(int *tab){
    for (int i=0; i<n; i++)
        printf("\t%d", *(tab+i));
}
void modifier(int tab[], int idx, int val){
    tab[idx] = val;
}
int main(){
    int T [] = {10, 22, 36, 50};
    modifier(&T[0], 2, 10);
    afficher(&T[0]);
    return 0;
}
```

4. Passage de paramètres avec les tableaux

- Dans le passage par adresse, on n'utilise pas le symbole `&` dans l'appel des fonctions car la variable de type tableau n'a pas une seule adresse comme le cas des variables de type simple.
- **Solution 2:** Faire l'appel au sous programme avec la variable T de type tableau

Passage par adresse

```
#include<stdio.h>
int n=4;
void afficher(int *tab){
    for (int i=0; i<n; i++)
        printf("\t%d", *(tab+i));
}
void modifier(int tab[], int idx, int val){
    tab[idx] = val;
}
int main(){
    int T [] = {10, 22, 36, 50};
    modifier(&T, 2, 10);
    afficher(&T);
return 0;
}
```

Erreur



Passage par adresse (Solution 2)

```
#include<stdio.h>
int n=4;
void afficher(int *tab){
    for (int i=0; i<n; i++)
        printf("\t%d", *(tab+i));
}
void modifier(int tab[], int idx, int val){
    tab[idx] = val;
}
int main(){
    int T [] = {10, 22, 36, 50};
    modifier(T, 2, 10);
    afficher(T);
return 0;
}
```

4. Passage de paramètres avec les tableaux

- Dans le passage par adresse, on n'utilise pas le symbole `&` dans l'appel des fonctions car la variable de type tableau n'a pas une seule adresse comme le cas des variables de type simple.
- **Solution 2:** Faire l'appel au sous programme la variable T de type tableau

Passage par adresse	Explication
<pre>#include<stdio.h> int n=4; void afficher(int *tab){ for (int i=0; i<n; i++) printf("\t%d", *(tab+i)); } void modifier(int tab[], int idx, int val){ tab[idx] = val; } int main(){ int T [] = {10, 22, 36, 50}; modifier(T, 2, 10); afficher(T); return 0; }</pre>	- tab : contient l'adresse du premier element du tableau T - *tab : contenu de l'adresse du premier element du tableau T - *tab $\Leftrightarrow$ tab[0] - tab+1 : adresse de l'element d'indice 1 dans le tableau T - tab+i : adresse de l'element d'indice i dans le tableau T - *(tab+i) = contenu de l'élément d'indice I dans le tableau T - *(tab+i) $\Leftrightarrow$ tab[i]

4. Passage de paramètres avec les tableaux

- **Exercice:** Ecrire un programme qui permet d'afficher les éléments et les adresses mémoires d'un tableau T en utilisant la notion des pointeurs

Passage par valeur	
<pre>#include<stdio.h> int main(){ int T [] = {10, 22, 36, 50}; printf("%d --> %x", *T, T); printf("\n%d --> %x", *(T+1), T+1); printf("\n%d --> %x", *(T+2), T+2); printf("\n%d --> %x", *(T+3), T+3); return 0; }</pre>	

Résultat	
10 --> 7b5ffc40	→
22 --> 7b5ffc44	→
36 --> 7b5ffc48	→
50 --> 7b5ffc4c	→

4. Passage de paramètres avec les tableaux

- **Exercice:** Ecrire un programme qui permet de remplir et d'afficher les éléments d'un tableau T de taille N en utilisant le passage par adresse

Solution

```
#include<stdio.h>
int N=4;
void afficher(int *tab){
    for (int i=0; i<N; i++)
        printf("\tT[%d]=%d", i, *(tab+i));
}
void remplir(int *tab){
    for (int i=0; i<N; i++){
        printf("T[%d]=", i);
        scanf("%d", tab+i);
    }
}
int main(){
    int T [N];
    remplir(T);
    afficher(T);
    return 0;
}
```